CodeWeaver comparison with RustRepoTrans

We selected one leakage-safe RustRepoTrans task per source language (C, Java, Python), ran three CodeWeaver repetitions, and evaluated each by replacing only the target function in a pristine licensed Rust project. Golden target bodies were hashed and excluded from every model-visible workspace. This 3/375-task slice measures feasibility and is not presented as a full-benchmark estimate.

Oracle calibration

The pristine goldens pass 284, 284, and 64 fixed tests. Replacing the selected functions with compiling panic stubs causes 50, 73, and at least 13 failures respectively, demonstrating non-vacuous coverage of each selected function.

Metric boundary

RustRepoTrans reports Pass@1 and one-round DSR@1 over 375 tasks. CodeWeaver is a multi-stage system with up to five repairs and three parity rounds. Its fixed-oracle pass-all rate is shown beside, but not relabeled as, Pass@1 or DSR@1.

Leakage audit

4/9 generated functions are byte-identical to the withheld golden body. This is disclosed as an output property, not evidence of exposure: every golden was hashed and removed before model access, and the actual workspace exclusion check is recorded in prepared metadata.

Redistribution

The RustRepoTrans repository has no visible repository-level license, so benchmark task text and full external projects are not redistributed. This artifact contains hashes, normalized outcomes, evaluation logs, and generated target functions under the target projects' MIT/Apache-2.0 licenses.

Measured stratified slice

Run	Pipeline terminal	Pass all	Build	Fixed tests	Test rate
CodeWeaver rep 1	3/3	3/3 (100.00%)	3/3 (100.00%)	632/632	100.00%
CodeWeaver rep 2	3/3	3/3 (100.00%)	3/3 (100.00%)	632/632	100.00%
CodeWeaver rep 3	3/3	3/3 (100.00%)	3/3 (100.00%)	632/632	100.00%

Per-language selected tasks

Source	Task	Build cells	Pass-all cells	Tests	Exact golden	Stub failures
C	incubator-milagro-crypto:RAND::clean	3/3	3/3	852/852	0/3	50

Source	Task	Build cells	Pass-all cells	Tests	Exact golden	Stub failures
Java	incubator-milagro-crypto:big::set	3/3	3/3	852/852	3/3	73
Python	charset-normalizer:CharsetMatch::encoding	3/3	3/3	192/192	1/3	13

Released artifact RQ1 / paper main-figure references

Model	Pass@1	DSR@1
DeepSeek-R1	51.50%	62.10%
DeepSeek-V3	50.10%	58.70%
Claude-3.5	43.50%	56.50%
Qwen-2.5-coder-32B	34.40%	38.90%

Complete source-paper surface audit

Surface	Denominator	Metrics	Artifact status
Table I	7 datasets	tokens, tasks, languages, dependencies, Rust, tests, verified goldens	structured_reference
Table II	7 models	type, openness, reasoning, release date, size	documented_reference
Figure 5	375 tasks	initial and self-debugging accuracy	structured_reference
Table III	300 versus 375 tasks	Pass@1 and DSR@1	structured_reference
RQ3	375 tasks	performance under dependency context	paper_only
Table IV	1,748 failed translations	compilation, runtime, functional, non-termination	structured_reference
Table V	1,614 compile-failing translations	top ten rustc codes and frequencies	structured_reference
Figures 6-8	1,610 categorized causes	10 causes in 3 supercategories; Cohen kappa 0.885	structured_reference
RQ5	released per-model result records	robustness and identification success	released_artifact_reference